

ECMPP Code User Guide

Environmentally Constrained Mine Planning Problem

May 29, 2026

 github.com/Eforay/ecmpp-mine-planning

 MIT License

 Python 3.8+

Abstract

This guide provides step-by-step instructions for downloading, installing, and running the ECMPP (Environmentally Constrained Mine Planning Problem) code from GitHub. The code implements a SAT/SMT-based approach to mine planning with environmental constraints, including a 25-block pedagogical example and scaling experiments. Following this guide, you will be able to reproduce the results reported in the paper “NP-Completeness of Mine Planning Under Logical Environmental Constraints”.

Contents

1	Overview	3
2	Step 1: Install Python	3
3	Step 2: Download the Code	3
3.1	Option A: Download as ZIP (Easiest)	3
3.2	Option B: Clone with Git (For Git Users)	3
4	Step 3: Open Command Prompt / Terminal	4
4.1	Windows	4
4.2	Mac / Linux	4
5	Step 4: Create a Virtual Environment (Recommended)	4
5.1	Create the environment	4
5.2	Activate it	4
6	Step 5: Install Dependencies	4
7	Step 6: Verify Installation	5
8	Step 7: Run the Pedagogical 25-Block Test	5
8.1	Expected Output	5
9	Step 8: Run Additional Tests	6
9.1	Clause Debugging	6
9.2	Scaling Experiments (Optional)	6
9.3	Interactive Session	6

10 Troubleshooting	8
11 Quick Reference: All Commands	8
12 Folder Structure	8
13 Interpreting Results	8
14 License	8
15 Citation	8

1 Overview

The ECMPP code repository contains:

- `ecmpp_encoder.py` — Z3 SMT encoding of the ECMPP decision problem
- `instance_generator.py` — Instance generator for pedagogical and synthetic problems
- `optimize.py` — Main optimization script (finds optimal admissible plan)
- `scaling_experiment.py` — Runs scaling experiments on synthetic instances
- `debug_clauses.py` — Debugging tool for clause parsing
- `25block_example/` — Pedagogical 25-block instance from Appendix A

2 Step 1: Install Python

If you don't have Python installed:

1. Go to python.org/downloads
2. Download Python **3.8 or higher** (3.11 or 3.12 recommended)
3. Run the installer
4. **IMPORTANT:** Check “Add Python to PATH” during installation

Verify installation:

```
python --version
```

3 Step 2: Download the Code

3.1 Option A: Download as ZIP (Easiest)

1. Go to <https://github.com/Efroy/ecmpp-mine-planning>
2. Click the green “Code” button
3. Select “Download ZIP”
4. Extract the ZIP file to a folder, e.g., `C:\Users\YourName\Downloads\ecmpp-mine-planning`

3.2 Option B: Clone with Git (For Git Users)

```
cd C:\Users\YourName\Downloads
git clone https://github.com/Eforay/ecmpp-mine-planning.git
cd ecmpp-mine-planning
```

4 Step 3: Open Command Prompt / Terminal

4.1 Windows

1. Press Windows Key, type cmd, press Enter
2. Navigate to the code folder:

```
cd C:\Users\YourName\Downloads\ecmpp-mine-planning
```

4.2 Mac / Linux

1. Open Terminal
2. Navigate to the code folder:

```
cd ~/Downloads/ecmpp-mine-planning
```

5 Step 4: Create a Virtual Environment (Recommended)

A virtual environment keeps dependencies organized and avoids conflicts.

5.1 Create the environment

```
python -m venv venv
```

5.2 Activate it

Activation Commands

Windows: venv\Scripts\activate
Mac/Linux: source venv/bin/activate

After activation, you should see (venv) at the beginning of the command line.

6 Step 5: Install Dependencies

Install required Python packages:

```
pip install -r requirements.txt
```

Expected output:

```
Successfully installed z3-solver-4.12.0 numpy-1.24.0 pandas-2.0.0
```

7 Step 6: Verify Installation

Run a quick test to ensure everything is working:

```
python -c "from instance_generator import ECMPPGenerator; g=ECMPPGenerator(); b,c,n=g.generate_pedagogical(); print(f'Success! Loaded {len(b)} blocks, {len(c)} clauses, NPV_min=${n}M')"
```

Expected output:

```
Success! Loaded 25 blocks, 8 clauses, NPV_min=$28M
```

8 Step 7: Run the Pedagogical 25-Block Test

Run the main optimization script:

```
python optimize.py
```

8.1 Expected Output

```
=====
ECMPP 25-Block Optimal Admissible Plan
=====

Loaded 25 blocks, 8 clauses
NPV minimum threshold: $28M

Adding precedence constraints...
  B6 requires B1
  B7 requires B2
  ... (more precedence constraints)

Adding environmental clauses...
Total clauses added: 8

Solving for optimal admissible plan...

[OPTIMAL SOLUTION]
  Status: SAT
  NPV: $37.1M
  Extracted blocks: B1, B2, B3, B5, B7, B8, B10, B12, B13,
  B15, B17, B18, B20, B23, B25
  Number of blocks: 15

Environmental predicates:
  E1 = True
  E2 = True
  E3 = True
  E4 = None
```

```
E5 = False
E6 = False
E7 = False
E8 = True

Acid drainage check:
  B17 extracted: True
  B22 extracted: False
  B23 extracted: True

Note: Different from manually constructed Plan C
  Plan C NPV: $31.1M
  Plan C blocks: 13
```

=====

9 Step 8: Run Additional Tests

9.1 Clause Debugging

Check how clauses are parsed:

```
python debug_clauses.py
```

9.2 Scaling Experiments (Optional)

```
python scaling_experiment.py
```

Note: This may take several minutes. Use **Ctrl+C** to cancel if needed.

9.3 Interactive Session

```
python
```

Then type:

```
from instance_generator import ECMPPGenerator
g = ECMPPGenerator()
blocks, clauses, npv_min = g.generate_pedagogical()
print(f"First block: {blocks[0]['id']} value=${blocks[0]['value']}M")
print(f"First clause: {clauses[0]}")
exit()
```

Table 1: Common Issues and Solutions

Problem	Solution
'python' is not recognized	Python not installed or not in PATH. Reinstall Python and check 'Add to PATH'.
ModuleNotFoundError: No module named 'z3'	Run <code>pip install z3-solver</code>
FileNotFoundError: [Errno 2]	You are not in the correct folder. Use <code>cd</code> to navigate to the code folder.
(venv) not showing	Activate virtual environment: <code>venv\Scripts\activate</code> (Windows) or <code>source venv/bin/activate</code> (Mac/Linux)
Slow or stuck solving	Press <code>Ctrl+C</code> to cancel. Large instances may time out.
ImportError: No module named 'pandas'	Run <code>pip install pandas</code>

Table 2: Command Quick Reference

Action	Command
Navigate to folder	<code>cd C:\Users\YourName\Downloads\ecmpp-mine-planning</code>
Activate virtual env (Windows)	<code>venv\Scripts\activate</code>
Activate virtual env (Mac/Linux)	<code>source venv/bin/activate</code>
Install dependencies	<code>pip install -r requirements.txt</code>
Test installation	<code>python -c "from instance_generator import ECMPPGenerator; print"</code>
Run main optimization	<code>python optimize.py</code>
Run clause debug	<code>python debug_clauses.py</code>
Run scaling test	<code>python scaling_experiment.py</code>
Deactivate virtual env	<code>deactivate</code>

10 Troubleshooting

11 Quick Reference: All Commands

12 Folder Structure

After downloading, your folder should look like:

```
ecmpp-mine-planning/  
  .gitignore  
  LICENSE  
  README.md  
  requirements.txt  
  __init__.py  
  ecmpp_encoder.py  
  instance_generator.py  
  optimize.py  
  scaling_experiment.py  
  debug_clauses.py  
  25block_example/  
    block_model.csv  
    clauses.txt
```

13 Interpreting Results

The optimal solution found by `optimize.py` should show:

- **Status: SAT** — a feasible admissible plan exists
- **NPV: \$37.1M** — the maximum net present value achievable while satisfying all constraints
- **Extracted blocks** — list of 15 blocks selected
- **Environmental predicates** — which environmental conditions are satisfied (E1, E2, E3, E8 true; E5, E6, E7 false)
- **Acid drainage check** — B22 is NOT extracted, satisfying pairwise acid drainage constraint C3b

This demonstrates that SAT-based optimization can identify admissible mine plans that manual heuristic approaches may miss, achieving a 19% higher NPV than the manually constructed Plan C (\$31.1M).

14 License

The code is released under the MIT License. See the `LICENSE` file in the repository for details.

15 Citation

If you use this code in your research, please cite:


```

@article{kudzawu2026npcompleteness,
  title={NP-Completeness of Mine Planning Under Logical Environmental Constraints},
  author={Kudzawu-D'Pherdd, Raymond and Nuviadenu, Johnson and
        Linus, Johanna H. and Ouabo, Emmanuel Romaric and
        Neil, Devan and Kudzawu-D'Pherdd, Elynam},
  journal={Computers \& Geosciences},
  year={2026},
  note={Under review}
}

```